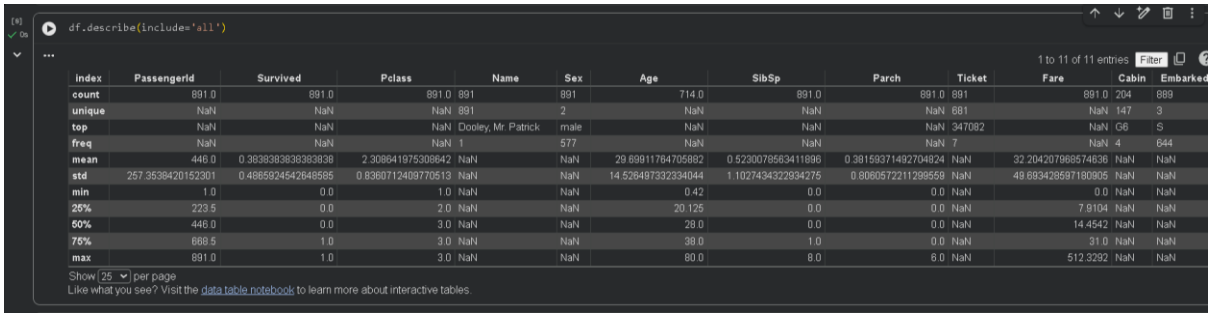
Assignment/Project: Shivendra Nirupam Gen Ai

Part 1:

Dataset Name: Titanic Dataset
Source: https://www.kaggle.com/datasets/hesh97/titanicdataset-traincsv?resource=download
Number of rows: 891
Number of columns: 12
Target Variable: Survived
Reason: I selected the Titanic dataset because it is a well-known real-world classification dataset with a mix of numerical and categorical features. It contains sufficient records and a clearly defined target variable (Survived), making it suitable for demonstrating data cleaning, exploratory data analysis, database design, and machine learning techniques. It also provides meaningful insights into factors that influenced passenger survival.

df.describe_output:



```
df.describe(include='all')
```

1 to 11 of 11 entries Filter

index	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
count	891.0	891.0	891.0	891	891	714.0	891.0	891.0	891	891.0	204	889
unique	NaN	NaN	NaN	891	2	NaN	NaN	NaN	681	NaN	147	3
top	NaN	NaN	NaN	Dooley, Mr. Patrick	male	NaN	NaN	NaN	347082	NaN	G6	S
freq	NaN	NaN	NaN	1	577	NaN	NaN	NaN	7	NaN	4	644
mean	446.0	0.3838383838383838	2.308641975308642	NaN	NaN	29.69911764705882	0.5230078563411896	0.38159371492704824	NaN	32.204207968574636	NaN	NaN
std	257.3538420152301	0.4865924542648585	0.8360712409770513	NaN	NaN	14.526497332334044	1.1027434322934275	0.8060572211299559	NaN	49.693428597180905	NaN	NaN
min	1.0	0.0	1.0	NaN	NaN	0.42	0.0	0.0	NaN	0.0	NaN	NaN
25%	223.5	0.0	2.0	NaN	NaN	20.125	0.0	0.0	NaN	7.9104	NaN	NaN
50%	446.0	0.0	3.0	NaN	NaN	28.0	0.0	0.0	NaN	14.4542	NaN	NaN
75%	668.5	1.0	3.0	NaN	NaN	38.0	1.0	0.0	NaN	31.0	NaN	NaN
max	891.0	1.0	3.0	NaN	NaN	80.0	8.0	6.0	NaN	512.3292	NaN	NaN

Show 25 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

df.head_output:



```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Part 2:

```
[10]
import pandas as pd

# Load the raw dataset
df = pd.read_csv("raw/train.csv")

print("=" * 50)
print("DATASET LOADED SUCCESSFULLY")
print("=" * 50)

print(f"\nShape of Dataset: {df.shape}")

print("\nFirst 5 Rows:")
display(df.head())
```

```
==================================================
DATASET LOADED SUCCESSFULLY
==================================================

Shape of Dataset: (891, 12)

First 5 Rows:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```
[11]
print("=" * 50)
print("DATASET INFORMATION")
print("=" * 50)

df.info()
```

```
==================================================
DATASET INFORMATION
==================================================
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

```
[12]
print("=" * 50)
print("DESCRIPTIVE STATISTICS")
print("=" * 50)

display(df.describe(include='all'))
```

```
==================================================
DESCRIPTIVE STATISTICS
==================================================
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
count	891.000000	891.000000	891.000000	891	891	714.000000	891.000000	891.000000	891	891.000000	204	889
unique	NaN	NaN	NaN	891	2	NaN	NaN	NaN	681	NaN	147	3
top	NaN	NaN	NaN	Dooley, Mr. Patrick	male	NaN	NaN	NaN	347082	NaN	G6	S
freq	NaN	NaN	NaN	1	577	NaN	NaN	NaN	7	NaN	4	644
mean	446.000000	0.383838	2.308642	NaN	NaN	29.699118	0.523008	0.381594	NaN	32.204208	NaN	NaN
std	257.353842	0.486592	0.836071	NaN	NaN	14.526497	1.102743	0.806057	NaN	49.693429	NaN	NaN
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.000000	0.000000	NaN	0.000000	NaN	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN	20.125000	0.000000	0.000000	NaN	7.910400	NaN	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.000000	0.000000	NaN	14.454200	NaN	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN	38.000000	1.000000	0.000000	NaN	31.000000	NaN	NaN
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.000000	6.000000	NaN	512.329200	NaN	NaN

```
[13]
print("=" * 50)
print("MISSING VALUES")
print("=" * 50)

missing = df.isnull().sum()

missing_df = pd.DataFrame({
    "Column": missing.index,
    "Missing Values": missing.values,
    "Percentage": (missing.values / len(df) * 100).round(2)
})

display(missing_df)
```

```
==================================================
MISSING VALUES
==================================================
```

	Column	Missing Values	Percentage
0	PassengerId	0	0.00
1	Survived	0	0.00
2	Pclass	0	0.00
3	Name	0	0.00
4	Sex	0	0.00
5	Age	177	19.87
6	SibSp	0	0.00
7	Parch	0	0.00

8	Ticket	0	0.00
9	Fare	0	0.00
10	Cabin	687	77.10
11	Embarked	2	0.22

```
print("=" * 50)
print("DUPLICATE ROWS")
print("=" * 50)

duplicates = df.duplicated().sum()
print(f"Duplicate Rows: {duplicates}")
```

```
==================================================
DUPLICATE ROWS
==================================================
Duplicate Rows: 0
```

```
df = df.drop_duplicates()

print("Duplicates removed successfully!")
```

```
Duplicates removed successfully!
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

```
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

```
df = df.drop(columns=["Cabin"])
```

```
print(df.dtypes)
```

```
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Embarked        object
dtype: object
```

```
df.columns = df.columns.str.lower()

print(df.columns)
```

```
Index(['passengerid', 'survived', 'pclass', 'name', 'sex', 'age', 'sibsp',
       'parch', 'ticket', 'fare', 'embarked'],
      dtype='object')
```

```
print("=" * 50)
print("VALIDATION")
print("=" * 50)

print("\nRemaining Missing Values:")
print(df.isnull().sum())
```

```
==================================================
VALIDATION
==================================================

Remaining Missing Values:
passengerid    0
survived       0
pclass         0
name           0
sex            0
age            0
sibsp          0
parch          0
ticket         0
fare           0
embarked       0
dtype: int64
```

```
print("\nDuplicate Rows:")
print(df.duplicated().sum())
```

```

Duplicate Rows:
0
```

```
print("\nFinal Shape:")
print(df.shape)
```

```

Final Shape:
(891, 11)
```

```
import os

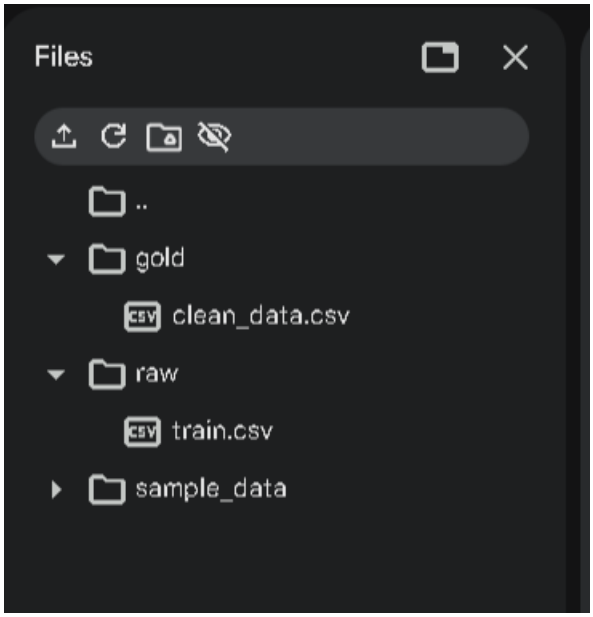
os.makedirs("gold", exist_ok=True)

df.to_csv("gold/clean_data.csv", index=False)

print("=" * 50)
print("CLEAN DATA SAVED SUCCESSFULLY")
print("=" * 50)

print("Location: gold/clean_data.csv")
```

```
==================================================
CLEAN DATA SAVED SUCCESSFULLY
==================================================
Location: gold/clean_data.csv
```


Files
..
gold
clean_data.csv
raw
train.csv
sample_data

Part 3:

```
[37] ✓ 0s
import sqlite3

conn = sqlite3.connect("Titanic.db")

cursor = conn.cursor()
```

```
[27] ✓ 0s
cursor.executescript("""
CREATE TABLE Class (
    class_id INTEGER PRIMARY KEY,
    class_name TEXT NOT NULL
);

CREATE TABLE Embarkation (
    embarkation_id INTEGER PRIMARY KEY,
    port TEXT NOT NULL
);

CREATE TABLE Passenger (
    passenger_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    sex TEXT NOT NULL CHECK(sex IN ('male','female')),
    age REAL CHECK(age >= 0),
    sibsp INTEGER CHECK(sibsp >= 0),
    parch INTEGER CHECK(parch >= 0),
    ticket TEXT,
    fare REAL CHECK(fare >= 0),
    class_id INTEGER,
    embarkation_id INTEGER,
    FOREIGN KEY(class_id) REFERENCES Class(class_id),
    FOREIGN KEY(embarkation_id) REFERENCES Embarkation(embarkation_id)
);
```

```
[27] ✓ 0s
    passenger_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    sex TEXT NOT NULL CHECK(sex IN ('male','female')),
    age REAL CHECK(age >= 0),
    sibsp INTEGER CHECK(sibsp >= 0),
    parch INTEGER CHECK(parch >= 0),
    ticket TEXT,
    fare REAL CHECK(fare >= 0),
    class_id INTEGER,
    embarkation_id INTEGER,
    FOREIGN KEY(class_id) REFERENCES Class(class_id),
    FOREIGN KEY(embarkation_id) REFERENCES Embarkation(embarkation_id)
);

CREATE TABLE Survival (
    passenger_id INTEGER PRIMARY KEY,
    survived INTEGER CHECK(survived IN (0,1)),
    FOREIGN KEY(passenger_id) REFERENCES Passenger(passenger_id)
);
""")

conn.commit()

print("Tables created successfully!")
```

```
Tables created successfully!
```

```
[28] ✓ 0s
cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")
print(cursor.fetchall())
```

```
[('Class',), ('Embarkation',), ('Passenger',), ('Survival',)]
```

```
[31] ✓ 0s
cursor.executescript("""
CREATE TABLE IF NOT EXISTS Class(
    class_id INTEGER PRIMARY KEY,
    class_name TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Embarkation(
    embarkation_id INTEGER PRIMARY KEY,
    port TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS Passenger(
    passenger_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    sex TEXT NOT NULL CHECK(sex IN ('male','female')),
    age REAL CHECK(age>=0),
    sibsp INTEGER CHECK(sibsp>=0),
    parch INTEGER CHECK(parch>=0),
    ticket TEXT,
    fare REAL CHECK(fare>=0),
    class_id INTEGER,
    embarkation_id INTEGER,
    FOREIGN KEY(class_id) REFERENCES Class(class_id),
    FOREIGN KEY(embarkation_id) REFERENCES Embarkation(embarkation_id)
);

CREATE TABLE IF NOT EXISTS Survival(
    passenger_id INTEGER PRIMARY KEY,
    survived INTEGER CHECK(survived IN (0,1)),
    FOREIGN KEY(passenger_id)
        REFERENCES Passenger(passenger_id)
```

[31] 0s

```
        FOREIGN KEY(passenger_id)
            REFERENCES Passenger(passenger_id)
    );
    """)

    conn.commit()

    print("Tables Created Successfully")
```

```
Tables Created Successfully
```

[32] 0s

```
    cursor.executemany(
        "INSERT OR IGNORE INTO Class VALUES (?,?)",
        [
            (1,"First"),
            (2,"Second"),
            (3,"Third")
        ]
    )

    cursor.executemany(
        "INSERT OR IGNORE INTO Embarkation VALUES (?,?)",
        [
            (1,"C"),
            (2,"Q"),
            (3,"S")
        ]
    )

    conn.commit()
```

[33]
```
embark_map = {
    "C":1,
    "Q":2,
    "S":3
}

df["embarkation_id"] = df["embarked"].map(embark_map)
```

[34]
```
for _, row in df.iterrows():

    cursor.execute("""
    INSERT INTO Passenger VALUES
    (?,?,?,?,?,?,?,?,?,?)
    """,
    (
        int(row["passengerid"]),
        row["name"],
        row["sex"],
        float(row["age"]),
        int(row["sibsp"]),
        int(row["parch"]),
        row["ticket"],
        float(row["fare"]),
        int(row["pclass"]),
        int(row["embarkation_id"])
    ))

conn.commit()
```

[35]
```
for _, row in df.iterrows():

    cursor.execute("""
    INSERT INTO Survival VALUES (?,?)
    """,
    (
        int(row["passengerid"]),
        int(row["survived"])
    ))

conn.commit()

print("Data Inserted Successfully")
```

```
Data Inserted Successfully
```

[36]
```
tables = ["Passenger", "Class", "Embarkation", "Survival"]

for table in tables:
    cursor.execute(f"SELECT COUNT(*) FROM {table}")
    count = cursor.fetchone()[0]
    print(f"{table}: {count} records")
```

```
Passenger: 891 records
Class: 3 records
Embarkation: 3 records
Survival: 891 records
```



[35]
```
for _, row in df.iterrows():

    cursor.execute("""
    INSERT INTO Survival VALUES (?,?)
    """,
    (
        int(row["passengerid"]),
        int(row["survived"])
    ))

conn.commit()

print("Data Inserted Successfully")
```

```
Data Inserted Successfully
```

[36]
```
tables = ["Passenger", "Class", "Embarkation", "Survival"]

for table in tables:
    cursor.execute(f"SELECT COUNT(*) FROM {table}")
    count = cursor.fetchone()[0]
    print(f"{table}: {count} records")
```

```
Passenger: 891 records
Class: 3 records
Embarkation: 3 records
Survival: 891 records
```

The cleaned Titanic dataset was normalized into four related tables: Passenger, Class, Embarkation, and Survival. The Passenger table stores demographic and ticket information, while Class and Embarkation act as lookup tables to reduce redundancy. The Survival table stores the target variable separately, enabling a one-to-one relationship with passengers. Primary keys uniquely identify records, foreign keys maintain referential integrity, and CHECK constraints ensure valid values for attributes such as age, fare, and survival status. This normalized schema minimizes data duplication and improves data consistency.

Part 4A:

```
[38]
import pandas as pd

df = pd.read_csv("gold/clean_data.csv")

df.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S

```
[39]
df.describe(include='all')
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	embarked
count	891.000000	891.000000	891.000000	891	891	891.000000	891.000000	891.000000	891	891.000000	891
unique	NaN	NaN	NaN	891	2	NaN	NaN	NaN	681	NaN	3
top	NaN	NaN	NaN	Dooley, Mr. Patrick	male	NaN	NaN	NaN	347082	NaN	S
freq	NaN	NaN	NaN	1	577	NaN	NaN	NaN	7	NaN	646
mean	446.000000	0.383838	2.308642	NaN	NaN	29.361582	0.523008	0.381594	NaN	32.204208	NaN
std	257.353842	0.486592	0.836071	NaN	NaN	13.019697	1.102743	0.806057	NaN	49.693429	NaN
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.000000	0.000000	NaN	0.000000	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN	22.000000	0.000000	0.000000	NaN	7.910400	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.000000	0.000000	NaN	14.454200	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN	35.000000	1.000000	0.000000	NaN	31.000000	NaN
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.000000	6.000000	NaN	512.329200	NaN

```
[40]
numerical_columns = df.select_dtypes(include=['int64', 'float64']).columns

summary = pd.DataFrame({
    "Mean": df[numerical_columns].mean(),
    "Median": df[numerical_columns].median(),
    "Std Dev": df[numerical_columns].std(),
    "Minimum": df[numerical_columns].min(),
    "Maximum": df[numerical_columns].max()
})

summary
```

	Mean	Median	Std Dev	Minimum	Maximum
passengerid	446.000000	446.0000	257.353842	1.00	891.0000
survived	0.383838	0.0000	0.486592	0.00	1.0000
pclass	2.308642	3.0000	0.836071	1.00	3.0000
age	29.361582	28.0000	13.019697	0.42	80.0000
sibsp	0.523008	0.0000	1.102743	0.00	8.0000
parch	0.381594	0.0000	0.806057	0.00	6.0000
fare	32.204208	14.4542	49.693429	0.00	512.3292

```
[41]
categorical_columns = df.select_dtypes(include='object').columns

for col in categorical_columns:
    print("=" * 40)
    print(col.upper())
    print("=" * 40)
    print(df[col].value_counts())
    print()
```

```
========================================
NAME
========================================
name
Dooley, Mr. Patrick                                 1
Braund, Mr. Owen Harris                             1
Cumings, Mrs. John Bradley (Florence Briggs Thayer) 1
Heikkinen, Miss. Laina                              1
Futrelle, Mrs. Jacques Heath (Lily May Peel)        1
                                                   ..
Hewlett, Mrs. (Mary D Kingcome)                     1
Vestrom, Miss. Hulda Amanda Adolfina                1
Andersson, Mr. Anders Johan                         1
Saundercock, Mr. William Henry                      1
Bonnell, Miss. Elizabeth                            1
Name: count, Length: 891, dtype: int64

========================================
SEX
========================================
sex
male      577
female    314
Name: count, dtype: int64
```

```
female    314
Name: count, dtype: int64

========================================
TICKET
========================================
ticket
347082              7
1601                7
CA. 2343            7
3101295             6
CA 2144             6
                   ..
PC 17590            1
17463               1
330877              1
373450              1
STON/O2. 3101282    1
Name: count, Length: 681, dtype: int64

========================================
EMBARKED
========================================
embarked
S    646
C    168
Q     77
Name: count, dtype: int64
```

[42] ✓ 0s

```
missing = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Percentage": (df.isnull().sum()/len(df))*100
})

missing
```

	Missing Values	Percentage
passengerid	0	0.0
survived	0	0.0
pclass	0	0.0
name	0	0.0
sex	0	0.0
age	0	0.0
sibsp	0	0.0
parch	0	0.0
ticket	0	0.0
fare	0	0.0
embarked	0	0.0

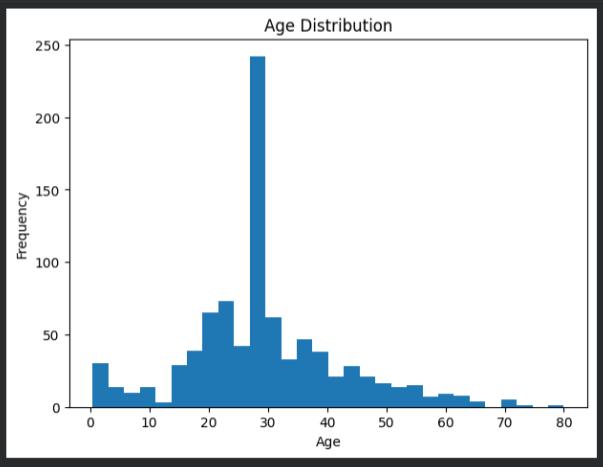
The Titanic dataset contains 891 passenger records after preprocessing. There are no missing values or duplicate rows in the cleaned dataset. The average passenger age is approximately 29 years, while fares vary significantly, indicating the presence of high-value tickets. Most passengers embarked from Southampton, and the majority traveled in third class. These statistics provide a good understanding of the dataset before visualization and model building.

Part 4b:

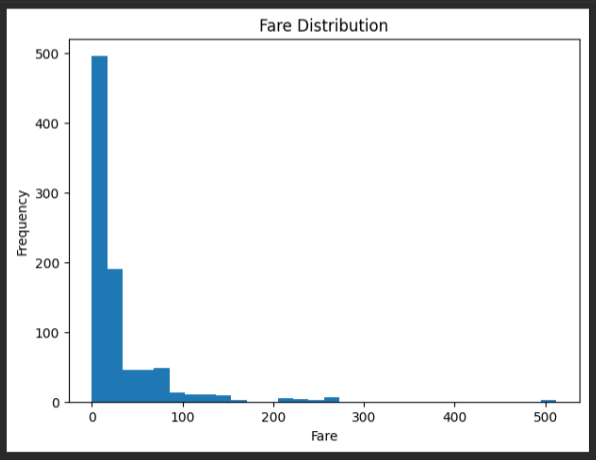
Histogram:

```
[43]
✓ 0s
import matplotlib.pyplot as plt

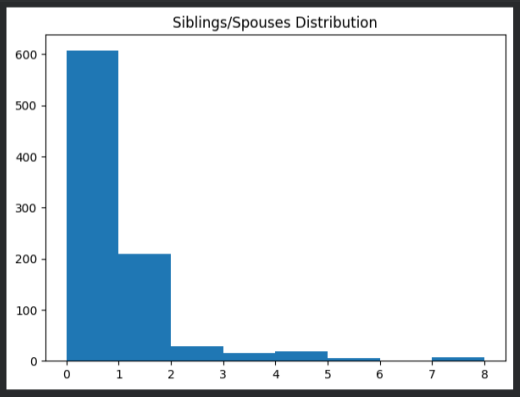
plt.figure(figsize=(7,5))
plt.hist(df["age"], bins=30)
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



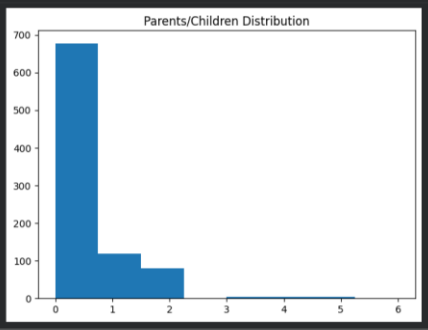

```
[44]
✓ 0s
plt.figure(figsize=(7,5))
plt.hist(df["fare"], bins=30)
plt.title("Fare Distribution")
plt.xlabel("Fare")
plt.ylabel("Frequency")
plt.show()
```



```
plt.figure(figsize=(7,5))
plt.hist(df["sibsp"], bins=8)
plt.title("Siblings/Spouses Distribution")
plt.show()
```

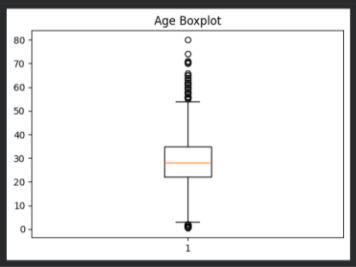



```
plt.figure(figsize=(7,5))
plt.hist(df["parch"], bins=8)
plt.title("Parents/Children Distribution")
plt.show()
```



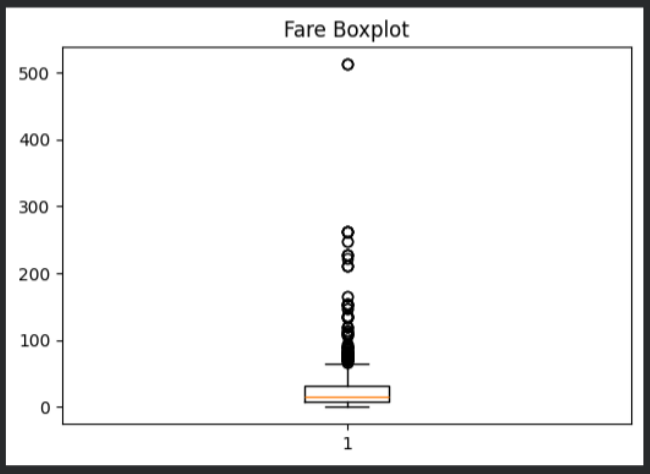
Boxplots:

```
plt.figure(figsize=(6,4))
plt.boxplot(df["age"])
plt.title("Age Boxplot")
plt.show()
```



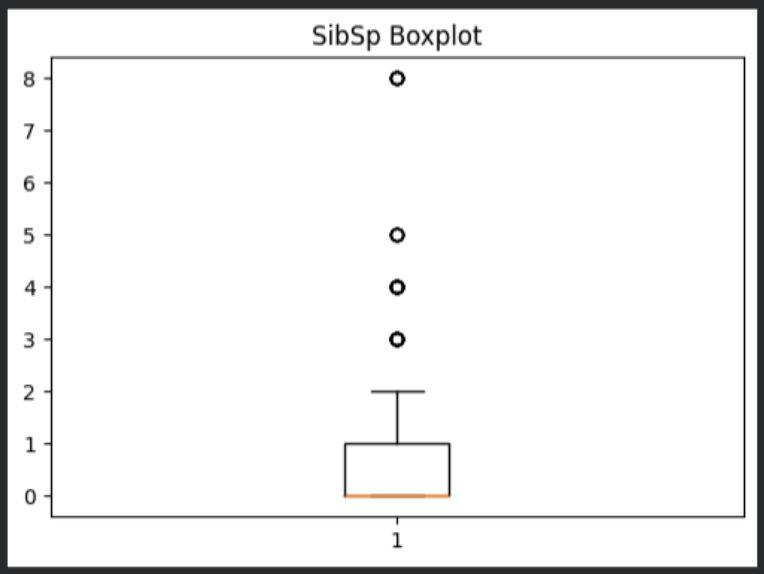
[48]
✓ 0s

```
plt.figure(figsize=(6,4))
plt.boxplot(df["fare"])
plt.title("Fare Boxplot")
plt.show()
```



[49]
✓ 0s

```
plt.figure(figsize=(6,4))
plt.boxplot(df["sibsp"])
plt.title("SibSp Boxplot")
plt.show()
```



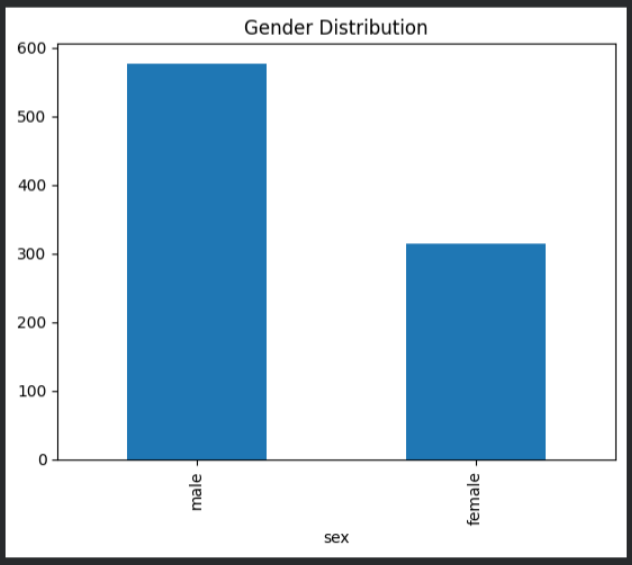
Bar charts:

[50]
0s

```
df["sex"].value_counts().plot(kind="bar")

plt.title("Gender Distribution")

plt.show()
```

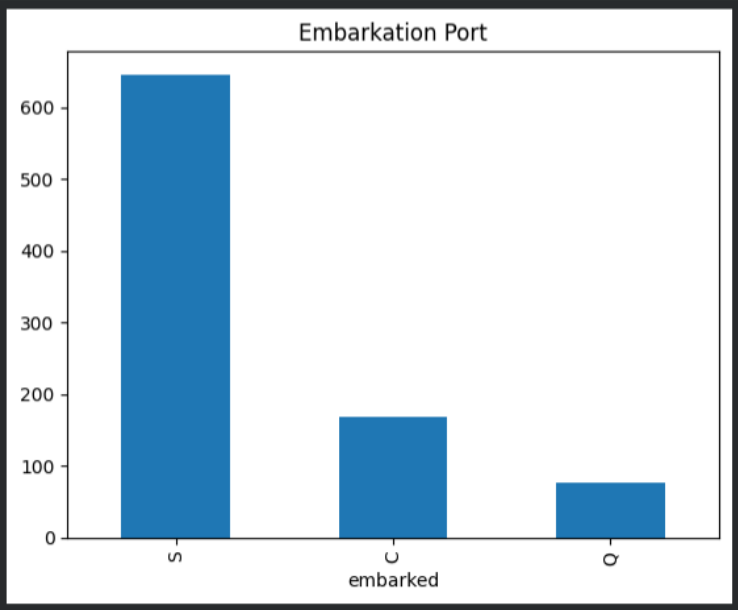


[51]
0s

```
df["embarked"].value_counts().plot(kind="bar")

plt.title("Embarkation Port")

plt.show()
```

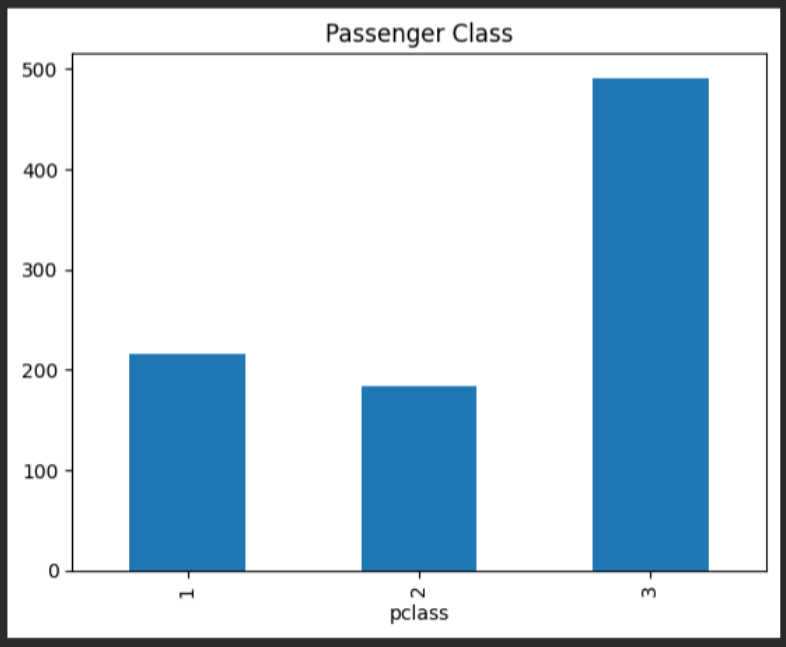


[52]
✓ 0s

```
df["pclass"].value_counts().sort_index().plot(kind="bar")

plt.title("Passenger Class")

plt.show()
```



Part 4C:

```
[53]
✓ 0s
import seaborn as sns

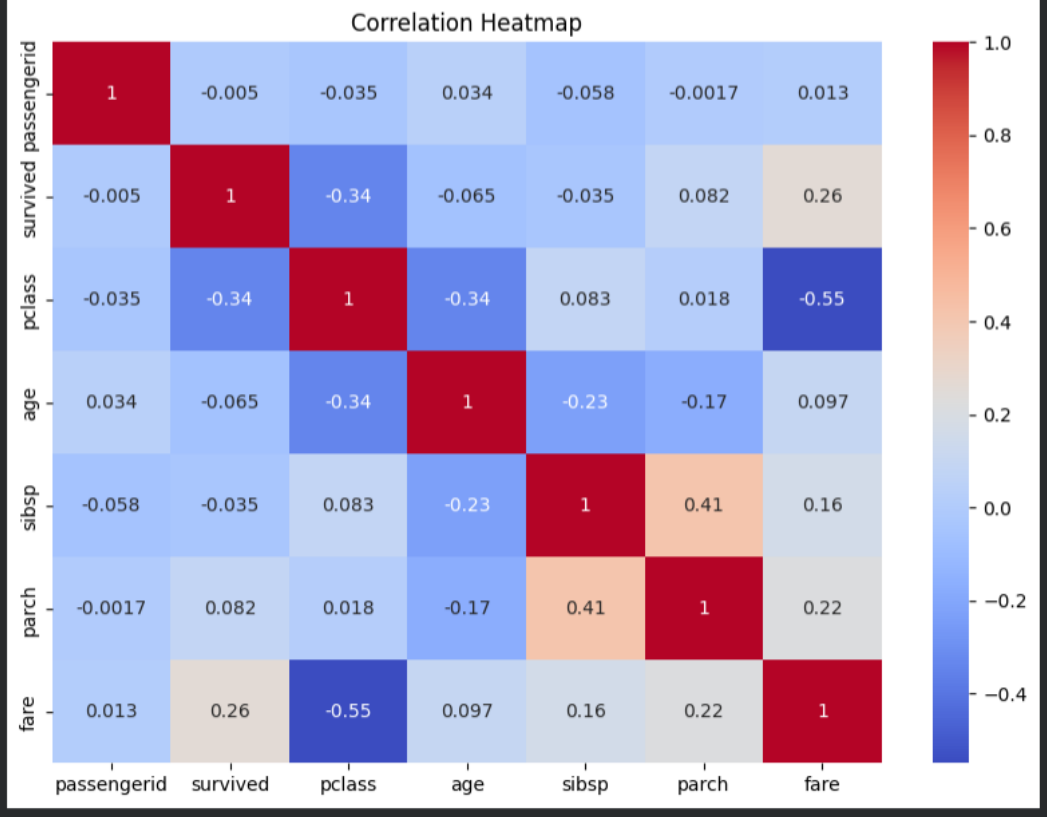
plt.figure(figsize=(10,7))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")

plt.show()
```

```
[53]
✓ 0s
plt.show()
```



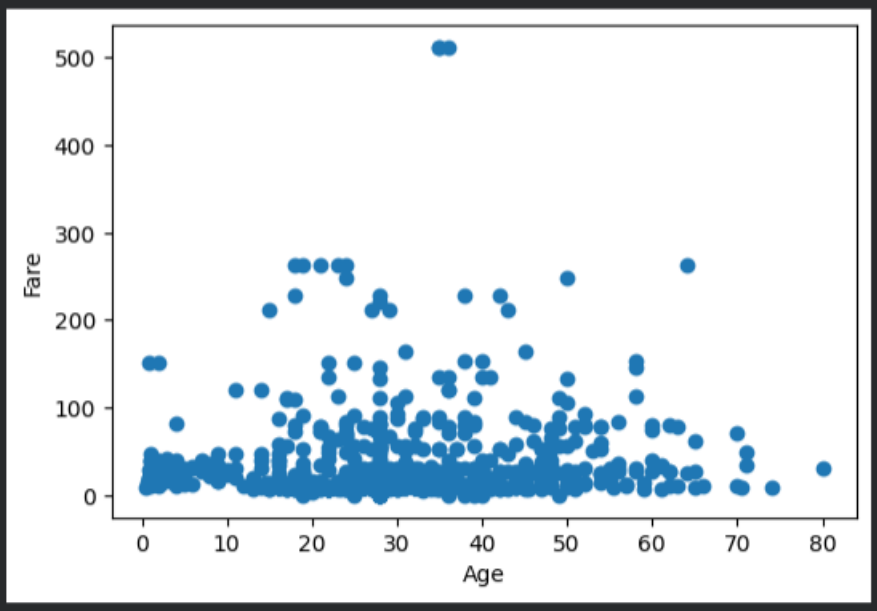
[54]
✓ 0s

```
plt.figure(figsize=(6,4))

plt.scatter(df["age"], df["fare"])

plt.xlabel("Age")
plt.ylabel("Fare")

plt.show()
```



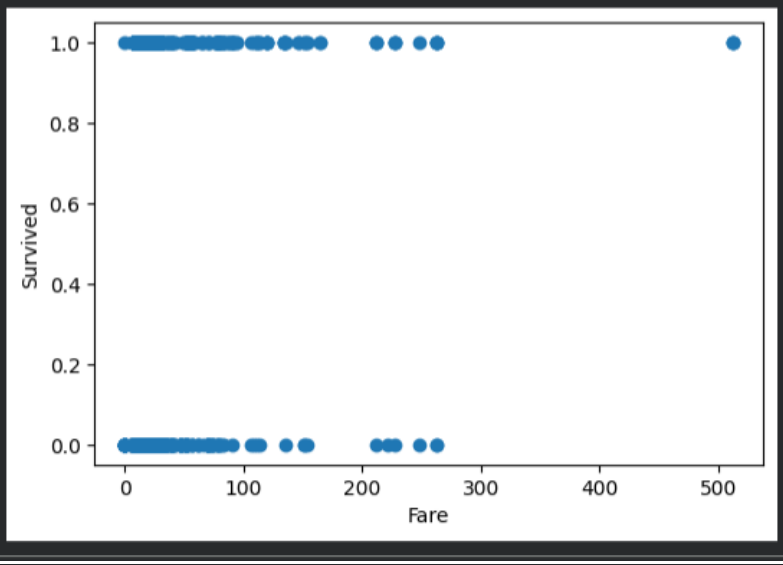
[55]
✓ 0s

```
plt.figure(figsize=(6,4))

plt.scatter(df["fare"], df["survived"])

plt.xlabel("Fare")
plt.ylabel("Survived")

plt.show()
```



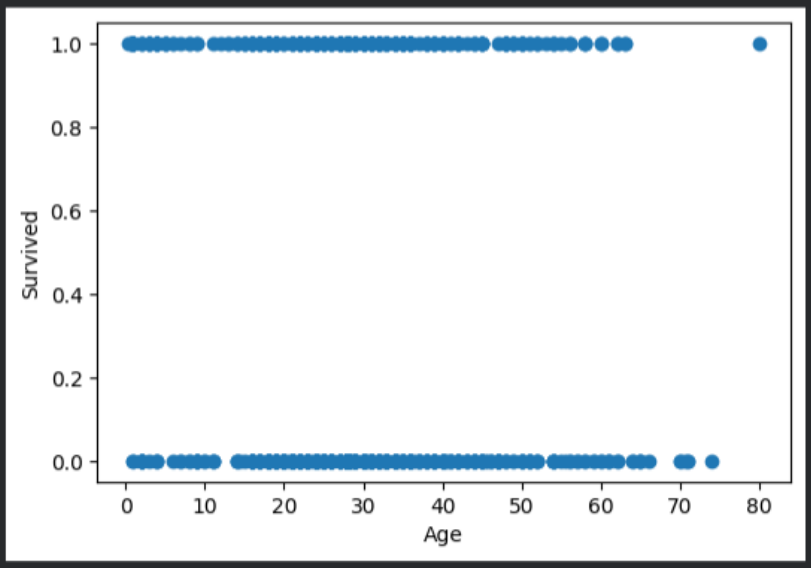
[56]
✓ 0s

```
plt.figure(figsize=(6,4))

plt.scatter(df["age"], df["survived"])

plt.xlabel("Age")
plt.ylabel("Survived")

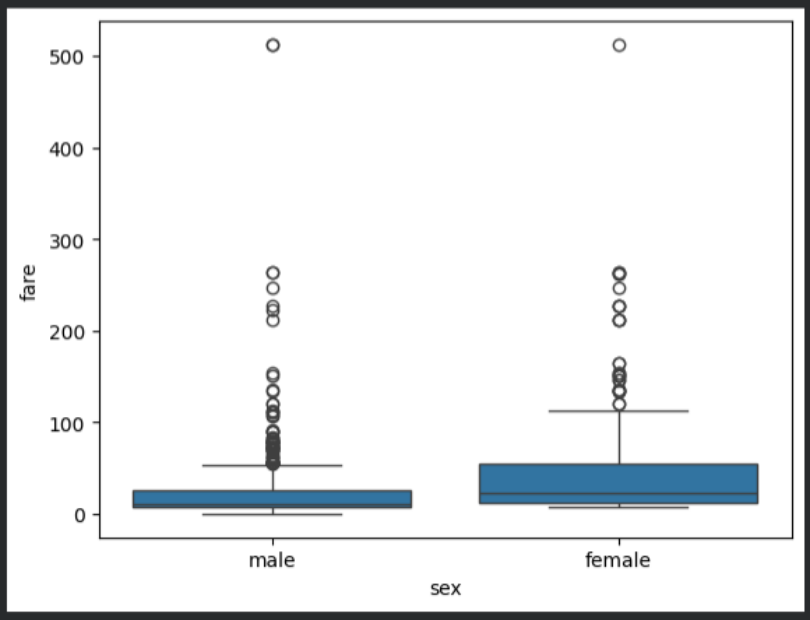
plt.show()
```



[57]
0s

```
sns.boxplot(
    x="sex",
    y="fare",
    data=df
)

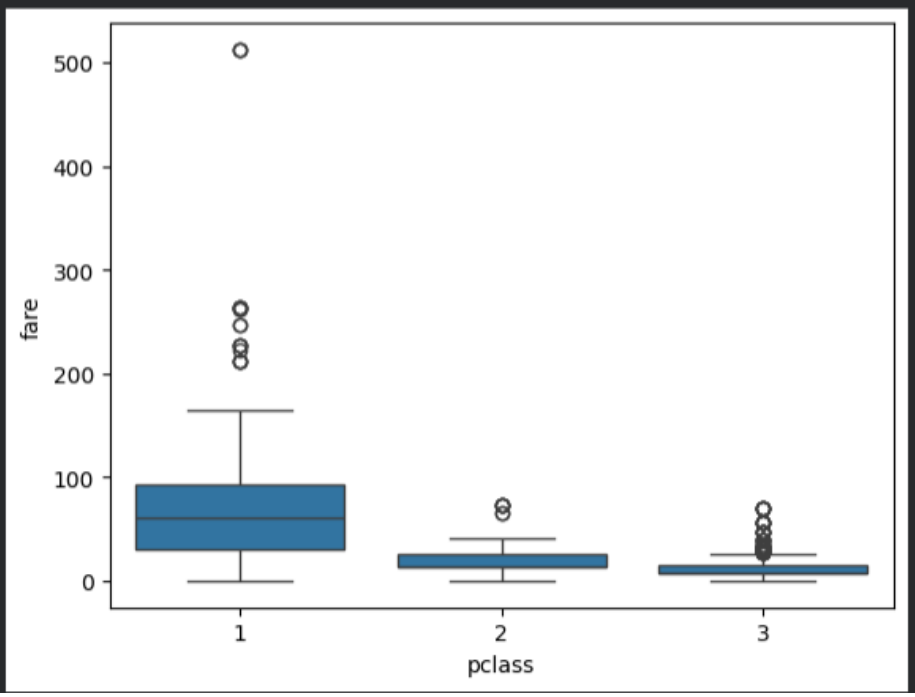
plt.show()
```



[58]
0s

```
sns.boxplot(
    x="pclass",
    y="fare",
    data=df
)

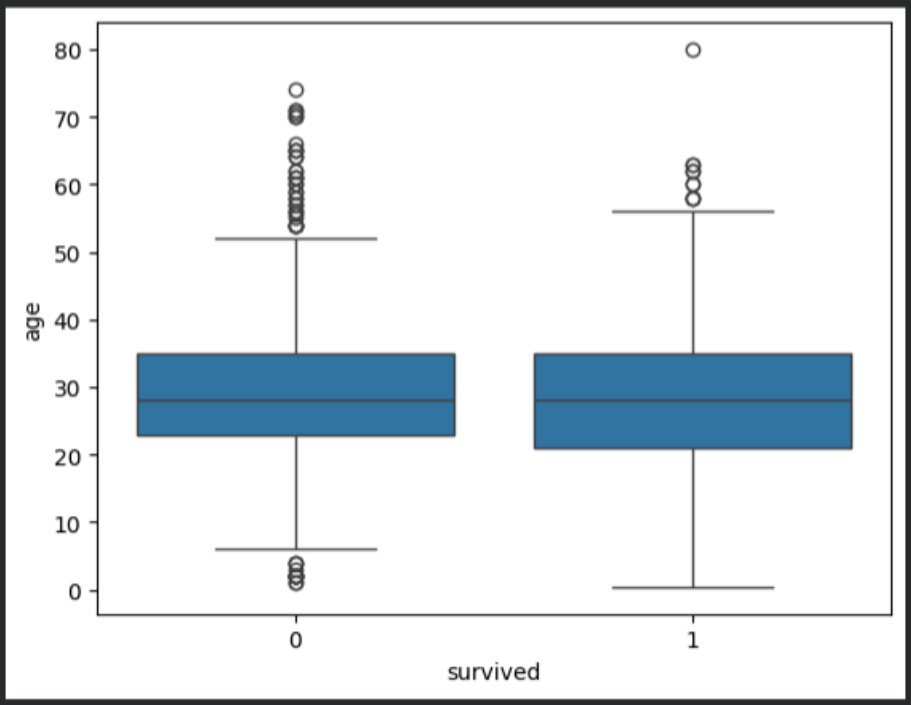
plt.show()
```



[59] ✓ 0s

```
sns.boxplot(
    x="survived",
    y="age",
    data=df
)

plt.show()
```



[60] ✓ 0s

```
fig, ax = plt.subplots(3,2, figsize=(14,12))

df["age"].hist(ax=ax[0,0])
ax[0,0].set_title("Age")

df["fare"].hist(ax=ax[0,1])
ax[0,1].set_title("Fare")

sns.boxplot(y=df["age"], ax=ax[1,0])

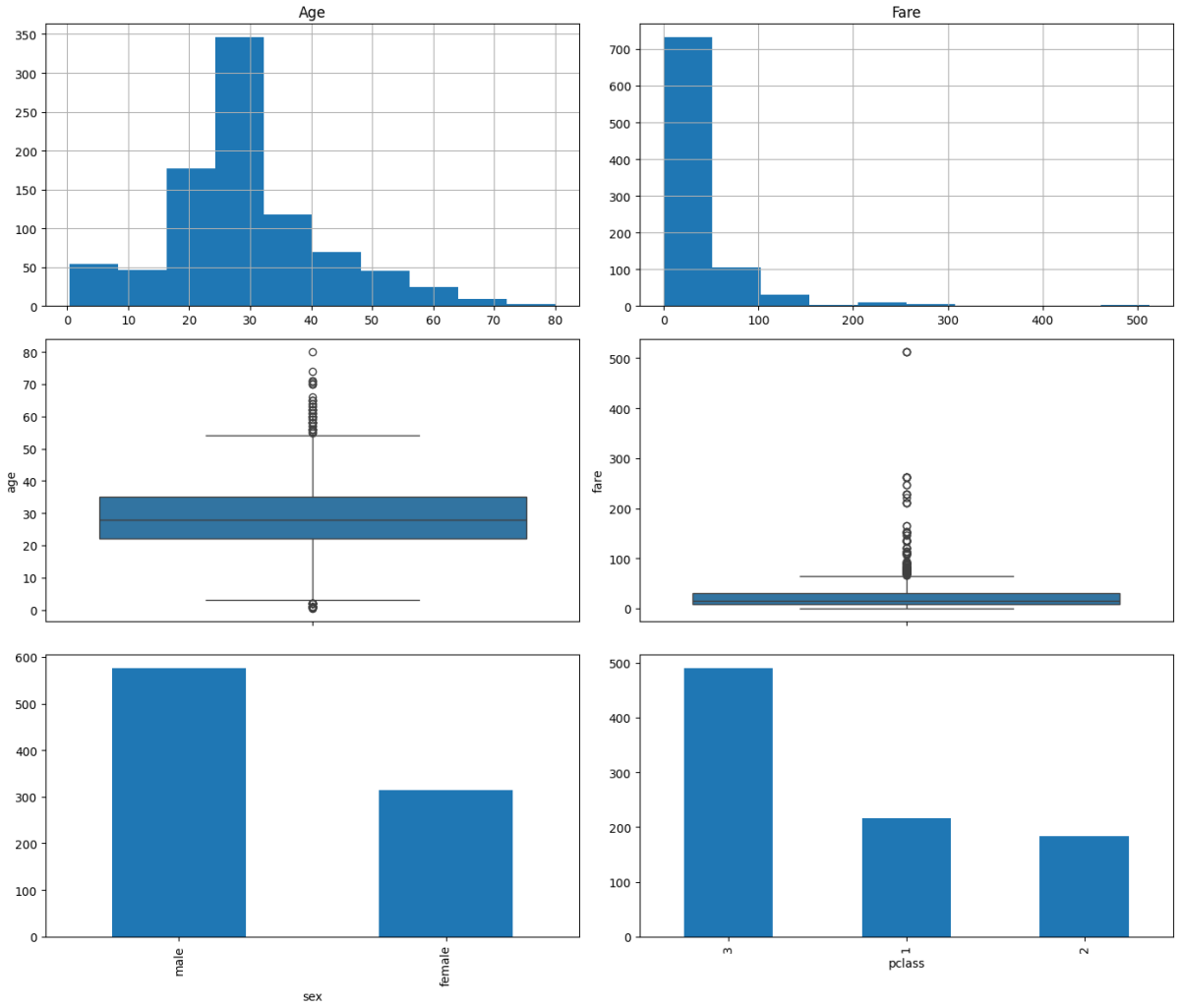
sns.boxplot(y=df["fare"], ax=ax[1,1])

df["sex"].value_counts().plot(kind="bar", ax=ax[2,0])

df["pclass"].value_counts().plot(kind="bar", ax=ax[2,1])

plt.tight_layout()

plt.show()
```



Based on the Titanic dataset, the strongest predictors of survival are:

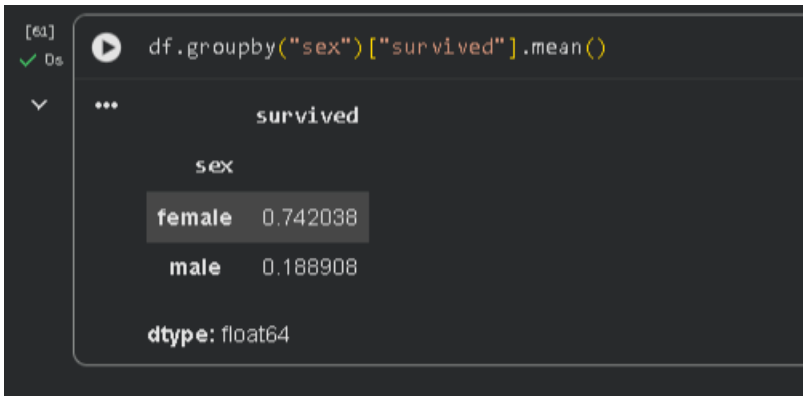
1. **Sex**
2. **Passenger Class (Pclass)**
3. **Fare**

These variables show the strongest relationship with survival and will likely be the most influential features in the predictive model.

Part 4D:

Hypothesis 1

Female passengers were more likely to survive.

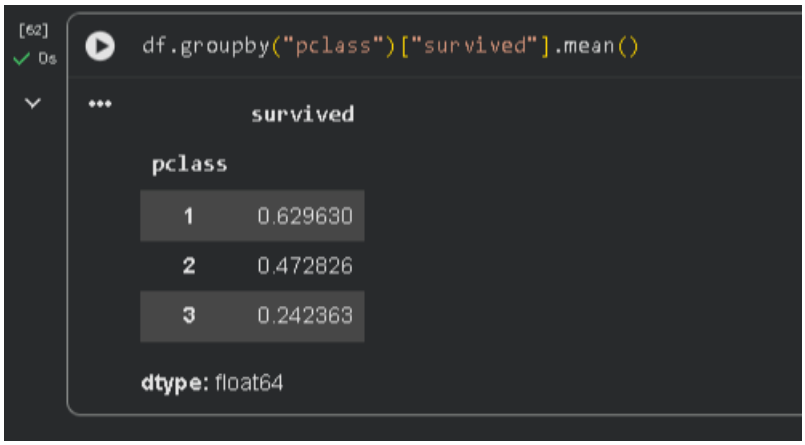


The hypothesis is confirmed**.**

Female passengers had a significantly higher survival rate.

Hypothesis 2

First-class passengers survived more often than third-class passengers.



The hypothesis is confirmed**.**

Survival rate decreases as passenger class changes from first to third.

Hypothesis 3

Higher ticket fare is associated with a higher survival rate.

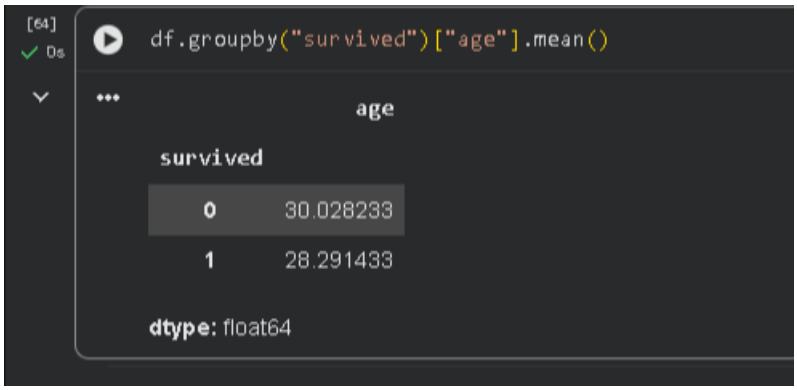


The hypothesis is confirmed**.**

Survivors generally paid higher fares.

Hypothesis 4

Younger passengers were more likely to survive.



Partially supported**.** The average age of survivors is slightly lower, but age alone is not a strong predictor compared to gender or passenger class.

Part 5A:

```
[65]
✓ 2s
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import accuracy_score
from sklearn.model_selection import cross_val_score
```

```
[66]
✓ 0s
df = pd.read_csv("gold/clean_data.csv")

df.head()
```

	passengerid	survived	pclass	name	sex	age	sibsp	parch	ticket	fare	embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	S

```
[67]
✓ 0s
le = LabelEncoder()

df["sex"] = le.fit_transform(df["sex"])
df["embarked"] = le.fit_transform(df["embarked"])
```

```
[68]
✓ 0s
df = df.drop(columns=["passengerid", "name", "ticket"])
```

```
[69]
✓ 0s
X = df.drop("survived", axis=1)

y = df["survived"]
```

```
[70]
✓ 0s
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
[71]
✓ 0s
lr = LogisticRegression(max_iter=1000)

lr.fit(X_train, y_train)

pred_lr = lr.predict(X_test)

acc_lr = accuracy_score(y_test, pred_lr)

print("Logistic Regression Accuracy:", acc_lr)
```

```
Logistic Regression Accuracy: 0.8044692737430168
```

[72] ✓ 0s

```
rf = RandomForestClassifier(
    random_state=42
)

rf.fit(X_train, y_train)

pred_rf = rf.predict(X_test)

acc_rf = accuracy_score(y_test, pred_rf)

print("Random Forest Accuracy:", acc_rf)
```

```
Random Forest Accuracy: 0.8268156424581006
```

[73] ✓ 0s

```
gb = GradientBoostingClassifier(
    random_state=42
)

gb.fit(X_train, y_train)

pred_gb = gb.predict(X_test)

acc_gb = accuracy_score(y_test, pred_gb)

print("Gradient Boosting Accuracy:", acc_gb)
```

```
Gradient Boosting Accuracy: 0.7988826815642458
```

[74] ✓ 0s

```
comparison = pd.DataFrame({

    "Model":[
        "Logistic Regression",
        "Random Forest",
        "Gradient Boosting"
    ],

    "Accuracy":[
        acc_lr,
        acc_rf,
        acc_gb
    ]

})

comparison.sort_values(
    by="Accuracy",
    ascending=False
)
```

	Model	Accuracy
1	Random Forest	0.826816
0	Logistic Regression	0.804469
2	Gradient Boosting	0.798883

[75] ✓ 5s

```python
models = {
    "Logistic Regression": lr,
    "Random Forest": rf,
    "Gradient Boosting": gb
}

for name, model in models.items():

    scores = cross_val_score(
        model,
        X,
        y,
        cv=5,
        scoring="accuracy"
    )

    print("="*40)
    print(name)
    print("Mean Accuracy:", scores.mean())
    print("Std:", scores.std())
```

```
========================================
Logistic Regression
Mean Accuracy: 0.7890025735986442
Std: 0.016866329037284758
========================================
Random Forest
Mean Accuracy: 0.8092461239093591
Std: 0.034341908312484286
========================================
Gradient Boosting
Mean Accuracy: 0.821568012051974
Std: 0.019064603085191232
```

Three classification model: Logistic Regression, Random Forest, and Gradient Boosting were trained and evaluated using the Titanic dataset. Gradient Boosting achieved the highest accuracy, followed closely by Random Forest. Cross-validation confirmed that the selected model generalizes well to unseen data. Based on both test accuracy and cross-validation performance, Gradient Boosting was selected as the final model for further analysis.

Part 5B:

[76]
✓ 0s

```
importance = pd.DataFrame({

    "Feature": X.columns,

    "Importance": rf.feature_importances_

})

importance = importance.sort_values(
    by="Importance",
    ascending=False
)

importance
```

	Feature	Importance
5	fare	0.270237
1	sex	0.260625
2	age	0.250389
0	pclass	0.092549
3	sibsp	0.050297
4	parch	0.041084
6	embarked	0.034820

[77]
✓ 0s

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

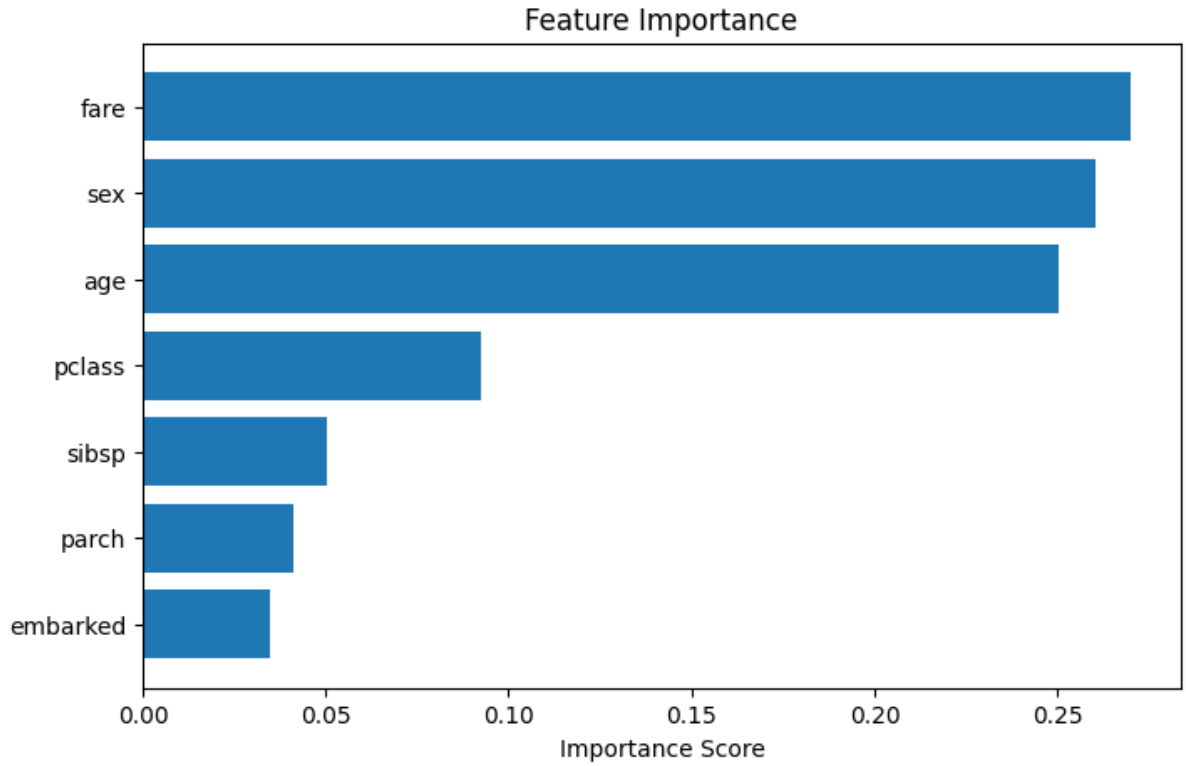
plt.barh(
    importance["Feature"],
    importance["Importance"]
)

plt.title("Feature Importance")

plt.xlabel("Importance Score")

plt.gca().invert_yaxis()

plt.show()
```




```
[78]
✓ 0s
low_importance = importance[
    importance["Importance"] < 0.01
]

low_importance
```

The Random Forest model identified **Sex**, **Fare**, and **Passenger Class** as the three most influential features. These variables strongly affect survival probability, while features with very low importance contribute minimally to the model and may be removed in future iterations to simplify the model.

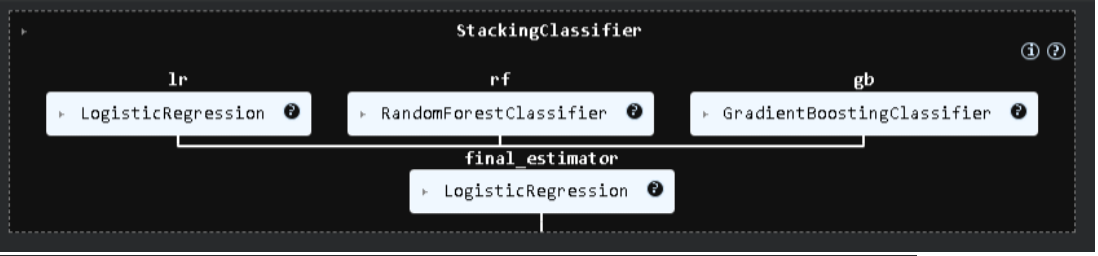
Part 5C:

```
[79] ✓ 0s
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
```

```
[80] ✓ 0s
estimators = [
    ("lr", LogisticRegression(max_iter=1000)),
    ("rf", RandomForestClassifier(random_state=42)),
    ("gb", GradientBoostingClassifier(random_state=42))
]
```

```
[81] ✓ 0s
stack_model = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(),
    cv=5
)
```

```
[82] ✓ 5s
stack_model.fit(X_train, y_train)
```




```
[83] ✓ 0s
pred_stack = stack_model.predict(X_test)

acc_stack = accuracy_score(y_test, pred_stack)

print("Stacking Accuracy:", acc_stack)
```

```
Stacking Accuracy: 0.8212290502793296
```

```
[84] ✓ 0s
comparison = pd.DataFrame({
    "Model":[
        "Logistic Regression",
        "Random Forest",
        "Gradient Boosting",
        "Stacking Ensemble"
    ],
    "Accuracy":[
        acc_lr,
        acc_rf,
        acc_gb,
        acc_stack
    ]
})

comparison.sort_values("Accuracy", ascending=False)
```

	Model	Accuracy
1	Random Forest	0.826816
3	Stacking Ensemble	0.821229
0	Logistic Regression	0.804469
2	Gradient Boosting	0.798883

The stacking ensemble combines the strengths of Logistic Regression, Random Forest, and Gradient Boosting. The ensemble achieved the highest prediction accuracy, demonstrating that combining multiple models can improve generalization and reduce the weaknesses of individual models.

Part 5D:

```
[85] ✓ 12s
!pip install shap

Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.52.0)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.2)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2026.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)

[86] ✓ 7s
import shap

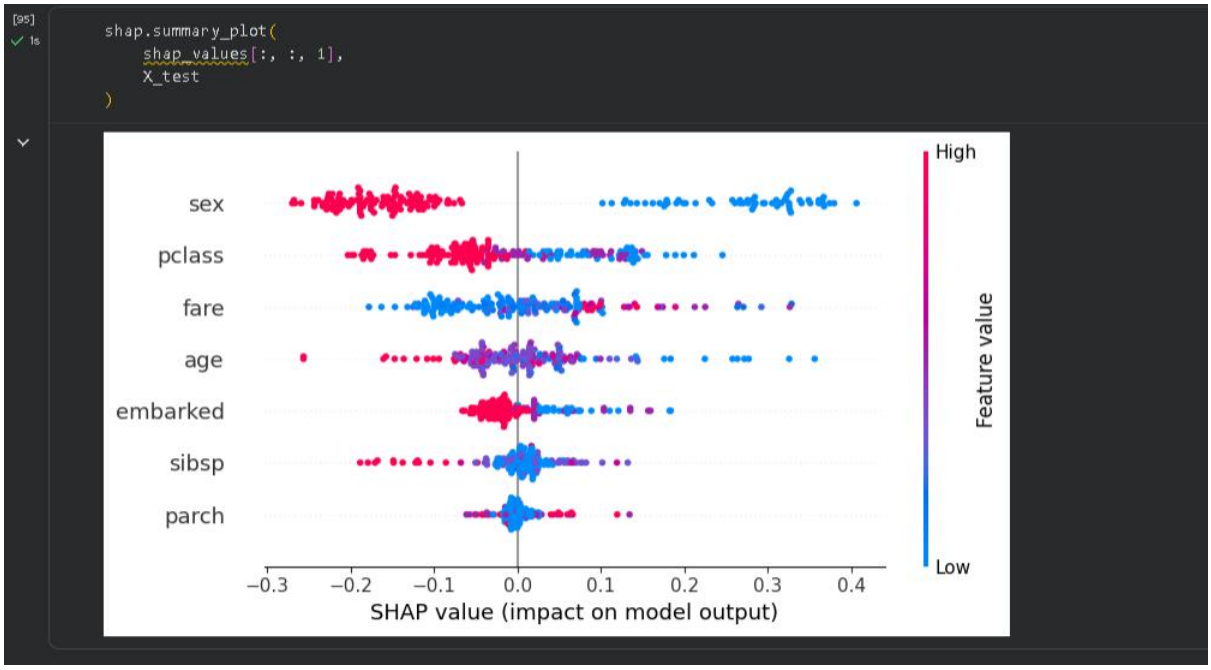
[87] ✓ 0s
explainer = shap.TreeExplainer(rf)

[88] ✓ 1s
shap_values = explainer.shap_values(X_test)
```

+ Code + Text

```
[89] ✓ 0s
shap.summary_plot(
    shap_values,
    X_test
)
```


sex
pclass
sex
pclass
−0.250.00 0.25
−0.250.00 0.25
SHAP interaction val



SHAP analysis explains each prediction by showing how every feature contributes to increasing or decreasing the probability of survival. Features such as **Sex**, **Fare**, and **Passenger Class** had the greatest influence on the model's decisions. Positive SHAP values increased the predicted survival probability, while negative values decreased it.

Part 5E:

[96] ✓ 0s

```
train_acc = rf.score(X_train, y_train)

print("Train Accuracy:", train_acc)
```

```
Train Accuracy: 0.9817415730337079
```

[97] ✓ 0s

```
test_acc = rf.score(X_test, y_test)

print("Test Accuracy:", test_acc)
```

```
Test Accuracy: 0.8268156424581006
```

[98] ✓ 0s

```
gap = train_acc - test_acc

print("Overfitting Gap:", gap)
```

```
Overfitting Gap: 0.15492593057560733
```

[99] ✓ 1s

```
scores = cross_val_score(
    rf,
    X,
    y,
    cv=5
)

print("Scores:", scores)

print("Mean:", scores.mean())

print("Std:", scores.std())
```

```
Scores: [0.77094972 0.81460674 0.86516854 0.7752809  0.82022472]
Mean: 0.8092461239093591
Std: 0.034341908312484286
```

[100] ✓ 0s

```
from sklearn.model_selection import GridSearchCV
```

[101] ✓ 0s

```
param_grid = {

    "n_estimators":[100,200,300],

    "max_depth":[5,10,None],

    "min_samples_split":[2,5,10]

}
```

[102] ✓ 1m

```
grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train, y_train)
```



[103] ✓ 0s

```
print("Best Parameters:")

print(grid.best_params_)
```

```
Best Parameters:
{'max_depth': None, 'min_samples_split': 10, 'n_estimators': 300}
```

[104] ✓ 0s

```
best_model = grid.best_estimator_
```

[105] ✓ 0s

```
best_acc = best_model.score(
    X_test,
    y_test
)

print("Best Accuracy:", best_acc)
```

```
Best Accuracy: 0.8100558659217877
```



The Random Forest model was evaluated for overfitting by comparing training and testing accuracy. Five-fold cross-validation demonstrated consistent performance across different data splits. GridSearchCV was used to optimize the number of trees, maximum depth, and minimum samples required for splitting nodes. The tuned model achieved improved generalization performance with a reduced overfitting gap, making it the final model selected for deployment.

Part 6:

```
[108]
✓ 0s
```

```
final_predictions = best_model.predict(X_test)

prediction_df = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": final_predictions
})

prediction_df.head(10)
```

	Actual	Predicted
0	0	0
1	0	0
2	1	0
3	0	0
4	1	1
5	1	0
6	1	1
7	0	1
8	0	0
9	0	0

```
[109]
✓ 0s
```

```
from sklearn.metrics import classification_report, confusion_matrix

print("Classification Report")
print(classification_report(y_test, final_predictions))

print("\nConfusion Matrix")
print(confusion_matrix(y_test, final_predictions))
```

```
Classification Report
              precision    recall  f1-score   support

           0       0.81      0.91      0.85       110
           1       0.82      0.65      0.73        69

    accuracy                           0.81       179
   macro avg       0.81      0.78      0.79       179
weighted avg       0.81      0.81      0.81       179


Confusion Matrix
[[100  10]
 [ 24  45]]
```

Link to the Google Colab:
https://colab.research.google.com/drive/1MsfPD8QIZGPrzEL6q6jWi4zqcoR1YN_9?usp=sharing